Model Selection for Prediction

Stat 230 - Spring 2026

Overview

In class, we built a model to *understand* the relationship between ridership, weekends, and holidays. In this activity, you will build models to **predict**: a bike-share company wants to forecast the number of `rides` on days it hasn't seen yet. They do not care about the relative importance of temperature/humidity/holidays/etc, only about how the model performs on predicting new days.

For prediction, a "good" model is one that does well on *new* data, not one that has the smallest p-values or the best fit to the data we already have.

We will compare six candidate models of increasing complexity. Run the code below to load the packages and the data, and set up the models.

> **i** Note:
>
> We're using some packages/functions we haven't seen before. Don't worry too much about these

```
library(dplyr)
library(purrr)
library(broom)
library(rsample)
library(yardstick)
library(parsnip)
library(workflows)
library(tune)

bikes <- read.csv("https://aluby.github.io/stat230/data/bikes.csv")
n <- nrow(bikes)

models <- list(
  M1 = rides ~ temp_actual,
  M2 = rides ~ temp_actual + humidity + windspeed,
  M3 = rides ~ temp_actual + humidity + windspeed + weekend + holiday + year,
  M4 = rides ~ temp_actual + humidity + windspeed + weekend + holiday + year +
season,
  M5 = rides ~ (temp_actual + humidity + windspeed + weekend + year +
season)^2,
  M6 = rides ~ (temp_actual + humidity + windspeed + weekend + year +
```

```
season)^3
)
```

Part 1: Judging models by the data used to fit them

```
m1 <- lm(rides ~ temp_actual, data = bikes)
m1_aug <- augment(m1)
rmse(m1_aug, truth = rides, estimate = .fitted)
```

```
# A tibble: 1 × 3
  .metric .estimator .estimate
  <chr>   <chr>          <dbl>
1 rmse    standard       1286.
```

```
bikes_fits <- map(models, \(f) lm(f, data = bikes))

full_rmse <- tibble(
  model = names(models),
  k = map_int(bikes_fits, \(m) length(coef(m))),
  rmse = map_dbl(bikes_fits, \(m) rmse(augment(m), truth = rides, estimate
= .fitted)$.estimate)
)
full_rmse
```

```
# A tibble: 6 × 3
  model     k  rmse
  <chr> <int> <dbl>
1 M1        2 1286.
2 M2        4 1185.
3 M3        7  792.
4 M4       10  662.
5 M5       34  609.
6 M6       74  550.
```

Q1. What happens to the RMSE as the models get more complex? Would you trust this table to tell you which model will predict *new* days best? Why or why not?

> The RMSE decreases every time, from about 1286 for M1 down to 550 for M6. This is guaranteed: adding terms can never make the fit to the *same* data worse. So this table would always crown the most complex model (M6), whether or not the extra terms capture real patterns or just noise in these particular 500 days. It measures how well the model *memorized* the data, not how well it will predict new days, so we can't trust it for choosing a prediction model.

Part 2: Training and test data

```
set.seed(75)

bikes_split <- initial_split(bikes, prop = 0.8)
bikes_split
```

```
<Training/Testing/Total>
<400/100/500>
```

```
bikes_train <- training(bikes_split)
bikes_test  <- testing(bikes_split)
```

Q2. Fill in the blanks so that each model is **fit on the training data** and then used to predict the **test data**. The code computes the RMSE on both datasets for every model. (For new data, `augment()` takes a `newdata` argument.) Record the results.

```
train_test_rmse <- map_dfr(models, \(f) {
  m <- lm(f, data = bikes_train)                        # fit on the TRAINING
data
  tibble(
    train = rmse(augment(m), truth = rides, estimate = .fitted)$.estimate,
    test  = rmse(augment(m, newdata = bikes_test), truth = rides, estimate
= .fitted)$.estimate  # predict the TEST data
  )
}, .id = "model")

train_test_rmse
```

```
# A tibble: 6 × 3
  model train  test
  <chr> <dbl> <dbl>
1 M1    1264. 1370.
2 M2    1165. 1265.
3 M3     789.  807.
4 M4     662.  683.
5 M5     600.  659.
6 M6     525.  697.
```

Q3. Answer the following using your table.

a. As the models get more complex, what happens to the **training** RMSE? What happens to the **test** RMSE?
b. Which model has the smallest test RMSE?

c. When a model is **overfit**, it fits the data used to build it very well but predicts new data poorly. Which model(s) appear to be overfit? Which appear to be **underfit** (too simple to capture the patterns in the data)? Explain using the gap between the training and test RMSE.

a. The training RMSE decreases with every model (from 1264 to 525), just like in Part 1. The test RMSE decreases at first (1370, 1265, 807, 683, down to 659 for M5), but then **goes back up** for M6 (697).

b. M5 has the smallest test RMSE (658.7).

c. M6 is overfit: its training RMSE is the smallest (525) but its test RMSE is worse than M5's and M4's, and the gap between test and training RMSE is about 172, compared with only about 21 for M4. It has so many terms (74 coefficients) that it is fitting noise specific to the training days. M5 is the "best fit to new data," though its gap (59) is already starting to grow. M1 and M2 are underfit: their errors are large on *both* datasets (about 1,200 to 1,400 rides), so they are missing important structure (they leave out `year`, `weekend`, and `season`, which carry a lot of the signal).

Part 3: One split is not enough

```
set.seed(230)
bikes_mc <- mc_cv(bikes, prop = 0.8, times = 100)

mc_rmse <- map_dfr(models, \(f) {
  workflow() |>
    add_model(linear_reg()) |>
    add_formula(f) |>
    fit_resamples(resamples = bikes_mc) |>
    collect_metrics(summarize = FALSE)
}, .id = "model") |>
  filter(.metric == "rmse")

# average and spread of the held-out RMSE across the 100 splits
mc_summary <- mc_rmse |>
  group_by(model) |>
  summarize(mean = mean(.estimate), sd = sd(.estimate))
mc_summary
```

```
# A tibble: 6 × 3
  model  mean    sd
  <chr> <dbl> <dbl>
1 M1    1289.  58.5
2 M2    1191.  59.7
3 M3     816.  64.2
4 M4     688.  77.9
```

```
5 M5     679.  81.5
6 M6     730. 186.
```

```
# how often was each model the best?
mc_best <- mc_rmse |>
  group_by(id) |>
  slice_min(.estimate) |>
  ungroup() |>
  count(model)
mc_best
```

```
# A tibble: 3 × 2
  model     n
  <chr> <int>
1 M4       35
2 M5       46
3 M6       19
```

Q4. Which model has the smallest *average* held-out RMSE? Compare M4 and M5: how large is the difference between them relative to the standard deviation of the RMSE across splits? Is there a clear winner?

> M5 has the smallest average held-out RMSE (678.5), followed closely by M4 (687.9). The difference between them is only about 9.4 rides, while the RMSE for either model varies by about 81 from split to split. So the difference is small relative to the noise from the random split, and there is **no clear winner** between M4 and M5. The counts show the same thing: M5 was best in 46 of the 100 splits, M4 in 35, and M6 in 19. What is clear is that M1 to M3 are too simple, and M6 is on average worse (730.1, with a much larger spread of 186), because it is overfit.

Q5. Is your answer to Q3(b) (from the single split) the same as your answer to Q4? What does this tell you about using a single train/test split to choose a model?

> Here, yes: both pick M5. But that is partly luck of the seed. M4 and M5 are close, so a different random split can pick M4 (or even M6; it was the best model in 19 of 100 splits). The single split's test RMSE for M5 (659) is also noticeably different from the average over many splits (679). A single split gives one noisy estimate of the prediction error, and it can easily change which model looks best when models are close. Averaging over many splits (or using cross-validation) gives a more stable answer.

```r
set.seed(75)
bikes_folds <- vfold_cv(bikes_train, v = 5)
bikes_folds
```

```
#  5-fold cross-validation
# A tibble: 5 × 2
  splits           id
  <list>           <chr>
1 <split [320/80]> Fold1
2 <split [320/80]> Fold2
3 <split [320/80]> Fold3
4 <split [320/80]> Fold4
5 <split [320/80]> Fold5
```

```r
cv_rmse <- map_dfr(models, \(f) {
  workflow() |>
    add_model(linear_reg()) |>
    add_formula(f) |>
    fit_resamples(resamples = bikes_folds) |>
    collect_metrics(summarize = TRUE)
}, .id = "model") |>
  filter(.metric == "rmse")

cv_rmse |> select(model, mean, std_err)
```

```
# A tibble: 6 × 3
  model  mean std_err
  <chr> <dbl>   <dbl>
1 M1    1270.    12.1
2 M2    1175.    30.3
3 M3     814.    29.2
4 M4     684.    40.7
5 M5     698.    69.2
6 M6    4014.  3296.
```

Q6. Which model does cross-validation prefer? Is the difference between the top two models clearly larger than the `std_err`? How does your answer compare with the repeated-split results in Q4? (You may notice a *huge* RMSE for one of the models. Explain what you think is happening.)

Cross-validation (using only the training data) prefers **M4** (683.9), followed by M5 (697.8). The difference between them (about 14 rides) is much smaller than the standard errors (41 and 69), so M4 and M5 are **essentially tied**. This is consistent with Q4, where M5 was slightly ahead on average but the difference was tiny compared with the split-to-split variability. (Notice that the single test split in Q3 favored M5. We should not use the test set to break

the tie, as we'll discuss in Q10; the model we pick should come from the training-data-only comparison, and a simpler model is the better tie-breaker.)

M6 has an enormous cross-validated RMSE (4014, with a std_err of 3296). Looking at the folds, one fold had an RMSE of about 17,192 while the other folds were 579, 584, 671, 1044. With 74 coefficients and only 320 training days per fold, the fitted M6 model is unstable. In one fold, it made wildly wrong predictions for a few unusual days. This is overfitting in its most dramatic form: a model so flexible that it can extrapolate wildly on new data.

Part 4: Compare to AIC and BIC

```
ic <- map_dfr(models, \(f) {
  glance(lm(f, data = bikes_train)) |>
    select(adj.r.squared, AIC, BIC)
}, .id = "model")

ic |>
  left_join(select(cv_rmse, model, cv_rmse = mean), by = "model")
```

```
# A tibble: 6 × 5
  model adj.r.squared   AIC   BIC cv_rmse
  <chr>         <dbl> <dbl> <dbl>   <dbl>
1 M1            0.339 6855. 6867.   1270.
2 M2            0.435 6794. 6814.   1175.
3 M3            0.739 6488. 6520.    814.
4 M4            0.815 6353. 6397.    684.
5 M5            0.838 6323. 6463.    698.
6 M6            0.861 6295. 6595.   4014.
```

Q7. Which model does AIC prefer? Which does BIC prefer? Compare with cross-validation. Which criterion's choice is closest to what cross-validation chose? What happened to M6 under each criterion, and why do you think BIC (which uses a bigger penalty) behaved differently from AIC?

AIC prefers M6 (6295.2, the smallest), even though M6 is the worst model by cross-validation. **BIC prefers M4** (6396.9), which is the same model that cross-validation chose. So BIC's choice matches cross-validation, while AIC's choice is the overfit model. (Adjusted R^2, which has no real penalty for complexity, also keeps increasing and picks M6.)

The reason is the penalty. AIC charges 2 per coefficient, which is cheap relative to the large drop in SSE that comes from adding 40 more terms, so M6 still looks good. BIC charges $\log(n) = \log(400) \approx 6$ per coefficient, so M6's 74 coefficients are penalized much more heavily (BIC for M6 is 6594.5 vs. 6396.9 for M4). So AIC is willing to add terms that BIC rejects. (Note that AIC is only *asymptotically* expected to track prediction error; with 400 days and 74 coefficients, we are not in that regime, which is why it overshoots here.)

Q8. Suppose two different people use this data:

- A bike-share company wants to **forecast** next week's ridership to decide how many bikes to put out.
- A city planner wants to **understand** how weekends and holidays affect ridership, after accounting for the weather.

Which of the six models (or what kind of model) would you recommend to each person? Do they necessarily need the same model? Explain, referring to your results above.

> **Forecasting:** I'd recommend M4. It has the smallest cross-validated RMSE (tied with M5 within the standard error) and is chosen by BIC, and it is much simpler than M5, so it is less at risk of overfitting. The goal is accurate predictions on new days, so we don't need to interpret every coefficient, and a more flexible model like M5 would also be defensible. We should avoid M6, which is overfit.
>
> **Understanding:** I'd recommend a smaller, interpretable model like M3 or M4 (these include `weekend` and `holiday`, and adjust for weather, `year`, and `season`). A model with dozens of interaction terms has no single "weekend effect" or "holiday effect" to interpret. (Each coefficient depends on the values of many other variables.) And since variables of interest like `weekend` and `holiday` should stay in the model, and inference is the goal, we pick based on the research question and interpretability, not just on prediction error.
>
> So they don't necessarily need the same model: the best model depends on the goal. A model that is best for prediction may be too complex to interpret, and a model that is good for interpretation may leave some predictive accuracy on the table. (In this example the answer happens to be similar, M4, because the simpler model predicts nearly as well as the complex one.)

Part 5: Selection and p-values

```
set.seed(5)

x <- matrix(rnorm(n * 20), nrow = n)   # 20 noise predictors
y <- rnorm(n)                          # a response unrelated to any of them

pvals <- map_dbl(1:20, \(j) tidy(lm(y ~ x[, j]))$p.value[2])
round(sort(pvals)[1:5], 4)   # the 5 smallest p-values
```

```
[1] 0.0858 0.1263 0.1752 0.1859 0.1939
```

```
set.seed(5)

min_p <- map_dbl(1:1000, \(i) {
  x <- matrix(rnorm(n * 20), nrow = n)
```

```
  y <- rnorm(n)
  min(map_dbl(1:20, \(j) tidy(lm(y ~ x[, j]))$p.value[2]))
})

mean(min_p < 0.05)
```

```
[1] 0.648
```

Q9. About what proportion of the time does the "best" of 20 pure-noise predictors have a p-value below 0.05? Explain why, using the fact that if each noise predictor has a 5% chance of looking "significant," the chance that *at least one of 20* does is $1 - 0.95^{20}$.

> About 0.65 (roughly 65%). Each noise predictor has a 5% chance of a p-value below 0.05 just by chance, so the chance that at least one of 20 does is $1 - 0.95^{20} = 0.642$. If we look at 20 variables and report the one with the smallest p-value, we will "find" a significant relationship about two-thirds of the time even when nothing is going on.

Q10. Based on Q9, explain why we should be skeptical of the p-values and confidence intervals from a model that was selected from many candidates by looking at the data. Then explain why we shouldn't use the *test set* to choose among many models and *also* report the test RMSE as our estimate of how well the chosen model will predict.

> **P-values after selection:** Standard p-values and confidence intervals assume the model was specified *before* looking at the data. If we compare many models or variables and keep the ones that look best, the terms that survive were selected *because* their p-values were small, so the actual Type I error rate is much larger than the nominal 5% (as in Q9), and the confidence intervals are too optimistic. The more models we try, the worse this gets. Selection tools like AIC, BIC, and RMSE help, but they don't fix this problem. This is why we should justify models with subject-matter knowledge, limit the number of candidate models, and be honest in the write-up about the selection process.
>
> **The test set:** The whole point of the test set is that the model never "saw" it. If we use the test RMSE to choose the winner among many models, the winner is partly the model that got *lucky* on that particular test set (the same selection effect as in Q9), so its test RMSE will be optimistic, and it will typically predict new data a bit worse than the test RMSE suggests. (In this activity, M5 had the smallest test RMSE in Q3 but cross-validation on the training data preferred M4. If we had picked M5 because of the test set, our reported test RMSE would be a little too good to be true.) To report an honest estimate of prediction error we need data that played no role in the selection: choose the model with cross-validation on the *training* data, then evaluate the chosen model on the test set **once**.

Function quick reference

Function	Purpose
`set.seed()`	Make random choices reproducible
`initial_split()`	Randomly split data into training and test sets
`training()`, `testing()`	Extract the training or test data from a split
`lm(formula, data)`	Fit a linear model
`augment(model, newdata)`	Add fitted values or predictions (`.fitted`) to a data frame
`rmse(data, truth, estimate)`	Root mean squared error
`glance()`	One-row summary of a model, including adjusted R^2, AIC, and BIC
`mc_cv()`, `vfold_cv()`	Create repeated random splits or v-fold cross-validation folds
`workflow()`, `add_model()`, `add_formula()`	Bundle a model type and a formula together
`fit_resamples()`, `collect_metrics()`	Fit a workflow to each split or fold and summarize the error
`map()`, `map_dfr()`	Repeat a computation for each model in a list